Stewer Gabriel

DOCUMENTAÇÃO DO SISTEMA

Referência da *API*

Rotas, formatos de pedido e resposta, erros e sessão.

Site e painel de stewergabriel.com
Visagismo e consultoria de imagem · Palmas, Tocantins · Desenvolvido por Gustavo Dev

Versão 2.1.0
08 de novembro de 2026

STEWER GABRIEL

Referência da API

Rotas, formatos de pedido e resposta, erros e sessão.

Todas as rotas ficam em `/api` e rodam nas Pages Functions, no mesmo endereço do site. O formato exato de cada resposta está nos schemas de `tests/contratos.ts`, conferidos a cada rodada de testes.

Convenções

- Pedidos com corpo mandam `Content-Type: application/json` (exceto envio de foto, que é `multipart/form-data`). Outro tipo dá **415**.
- Corpo grande demais dá **413** (64 KB no geral, 32 KB nos textos, 2 KB em login, senha e contatos).
- Todo erro sai como `{ "erro": "mensagem em português" }`, pronta para mostrar na tela. Erro inesperado sai como 500 sem detalhe interno.
- Nenhuma resposta da API fica em cache (`Cache-Control: no-store`), exceto as fotos e `/conteudo` (`no-cache`: o navegador sempre confere com o servidor, mas pode reaproveitar o `<link rel="preload">` do `index.html`).
- Toda resposta leva os cabeçalhos de segurança (`nosniff`, `X-Frame-Options: DENY`, CSP `default-src 'none'`), inclusive erro e foto.
- Rota que não existe responde **404** (ou **405** para método diferente de GET) em JSON, nunca a página do site.
- Datas vêm do D1 em UTC, no formato `2026-11-07 22:30:00`.

STATUS	QUANDO
400	dado inválido (a mensagem diz qual)
401	sem sessão, sessão vencida ou senha errada
403	outra origem (CSRF), ou senha temporária ainda não trocada
404	rota, lista ou item que não existe (id de rota só com dígitos)
405	método que a rota não aceita
409	conflito: alguém salvou antes, foto em uso, lista mudou ao reordenar
413 / 415	corpo grande demais / tipo errado
429	login travado depois de 5 erros
502	o Google não respondeu ao abrir o link curto do mapa

Público

`GET /conteudo`

Tudo que o site mostra, numa ida só ao banco. Só itens visíveis, na ordem do painel, sem ids, ordem nem versão.


```
{
  "slides":   [{ "src": "/photos/Stewer1.jpg", "alt": "Stewer Gabriel", "largura": 1024,
"altura": 1024, "posicao": "50% 0%" }],
  "numeros":  [{ "valor": "+800", "legenda": "Clientes atendidos por ano" }],
  "servicos": [{ "titulo": "Corte", "texto": "Cada corte é…" }],
  "etapas":   [{ "titulo": "Análise de Visagismo", "texto": "…" }],
  "topicos":  [{ "titulo": "Técnicas de Corte", "texto": "…" }],
  "aulas":    [{ "src": "/instrutorImgs/instrutor2.jpeg", "alt": "…", "largura": 960, "altura":
1280, "posicao": "50% 50%" }],
  "cortes":   [{ "src": "/api/midias/12", "alt": "Degradê com risca", "largura": 1200,
"altura": 1600, "posicao": "50% 50%" }],
  "config":   { "whatsapp": "5563985116783", "instagram": "stewer.gabriel", "mensagem": "Olá
Stewer, …", "latitude": "-10.18256053", "longitude": "-48.32910908" },
  "textos":   { "hero.lema": "Novo lema", "sobre.foto": { "src": "/api/midias/9", "alt": "…",
"largura": 800, "altura": 1000 } }
}
```


`textos` só traz os campos trocados no painel; o resto o site já tem.

`GET /midias/:id`

A foto. Foto enviada pelo painel sai com o tipo real, `nosniff` e cache de um ano (`immutable`). Foto original redireciona (302) para o arquivo em `public/`. `?baixar=1` entrega como download, com nome legível tirado da descrição (`degrade-com-risca-12.webp`).

Sessão

`POST /auth/login`

`{ "email", "senha" }` → `{ "usuario": { id, nome, email, trocarSenha } }` e o cookie `sg_sessao`. Senha errada e e-mail que não existe dão a mesma resposta (401). Depois de 5 erros no mesmo e-mail, 429 por 15 minutos.

O cookie leva um JWT (HS256, 8 horas) que aponta para uma linha da tabela `sessoes`. Sem a linha, o token não vale, por melhor que seja a assinatura.

`POST /auth/logout`

Apaga o cookie e encerra a sessão no servidor: um token copiado antes deixa de valer. As sessões da mesma pessoa em outros aparelhos continuam.

`GET /auth/eu`

Quem está logado, ou 401. O painel usa ao abrir.

Painel (`/admin`, exige sessão)

Toda rota de `/api/admin` passa pelo `_middleware.ts`: sem sessão, 401; pedido que muda dados vindo de outra origem, 403; pessoa removida perde o acesso na hora; senha temporária só deixa trocar a senha.

Listas: `slides`, `numeros`, `servicos`, `etapas`, `topicos`, `aulas`, `cortes`

MÉTODO	ROTA	O QUE FAZ
GET	`/admin/:lista`	tudo, inclusive oculto, na ordem
POST	`/admin/:lista`	cria no fim; responde 201 com o item
PUT	`/admin/:lista/:id`	altera só os campos enviados
DELETE	`/admin/:lista/:id`	apaga
PUT	`/admin/:lista/ordem`	`{ "ids": [3, 1, 2] }`, todos os itens, numa transação

Campos de cada lista:

LISTA	CAMPOS	LIMITES
`servicos`	`titulo`, `texto`, `visivel`	40 / 300
`etapas`	`titulo`, `texto`, `visivel`	50 / 300
`topicos`	`titulo`, `texto`, `visivel`	50 / 200
`numeros`	`valor`, `legenda`, `visivel`	10 / 40
`slides`, `aulas`, `cortes`	`foto_id`, `foto_posicao` (`"50% 30%"`), `visivel`	foto precisa existir

No PUT, o painel manda também `versao` (a que tinha ao abrir). Se não bater com a do banco, 409 com o nome de quem salvou antes. Sem `versao` (o interruptor "No site"), vale o último.

Nas listas de foto, o GET e o item devolvido trazem `foto_url` e `foto_alt`, e o nome do item na auditoria é a descrição da foto.

A ordem precisa ter exatamente os itens que existem; se alguém criou ou apagou um item no meio, 409.

Fotos

MÉTODO	ROTA	O QUE FAZ
GET	`/admin/midias`	a biblioteca, sem os bytes, com quantos lugares usam cada foto
POST	`/admin/midias`	multipart: `arquivo`, `alt`, `largura`, `altura`
PUT	`/admin/midias/:id`	`{ "alt" }` corrige a descrição
DELETE	`/admin/midias/:id`	apaga se nada usar; foto original nunca

Upload: só JPG, PNG e WebP, conferidos pelos primeiros bytes, até 1,5 MB. Descrição de 3 a 200 caracteres.

Textos do site

`GET /admin/textos` → `{ valores: { chave: valor }, fotos: { chave: url } }`

`PUT /admin/textos` com `{ "valores": { "hero.lema": "…", "sobre.foto": "12", "rodape.frase": null } }`. `null` volta o campo ao original. As chaves válidas e os limites estão em `src/lib/textos.ts`. Um campo inválido barra o pacote inteiro. Em campo de uma linha, quebra de linha vira espaço; em campo de várias linhas, linhas vazias somem.

Contatos e mapa

`GET /admin/configuracoes` → `{ config: { whatsapp, instagram, mensagem, latitude, longitude } }`

`PUT /admin/configuracoes` com qualquer combinação de:

CAMPO	ACEITA	GRAVA
`whatsapp`	`(63) 98511-6783`, com ou sem 55	`5563985116783`
`instagram`	`@stewer.gabriel` ou o link do perfil	`stewer.gabriel`
`mensagem`	3 a 200 caracteres	espaços e quebras juntados
`coordenadas`	`-10.18256, -48.32910` como o Google Maps copia, ou o link longo de um lugar	`latitude` e `longitude`, só se o ponto for no Brasil

Tudo junto ou nada.

`POST /admin/link-mapa` `{ url }` → `{ latitude, longitude }`: abre um link do Google Maps e devolve o ponto. Link longo é lido direto, sem rede. Link curto (`maps.app.goo.gl`, do botão Compartilhar) é seguido pelo servidor: só para hosts do Google, só HTTPS, sem usuário nem porta, até 6 redirecionamentos e 5 s. Qualquer outro endereço dá 400 sem nenhum acesso (proteção contra SSRF).

Usuários

MÉTODO	ROTA	O QUE FAZ
GET	`/admin/usuarios`	quem tem acesso, com o último login
POST	`/admin/usuarios`	`{ nome, email }` → item e `senhaTemporaria` (aparece uma vez)
PUT	`/admin/usuarios/:id`	gera senha temporária nova (esqueceu a senha) e encerra as sessões abertas da pessoa
DELETE	`/admin/usuarios/:id`	tira o acesso; não vale para si mesmo nem para o último

Outros

MÉTODO	ROTA	O QUE FAZ
PUT	`/admin/senha`	`{ atual, nova }`; quem está com senha temporária não precisa da atual. Mínimo 10 caracteres. Encerra as outras sessões da pessoa
GET	`/admin/auditoria?de=…&ate=…[&usuario=id]`	registros do período, do mais novo ao mais velho (`usuario` que não é número dá 400)
GET	`/admin/espaco`	`{ usado, limite, porcentagem, bytesFotos, fotosEnviadas }`